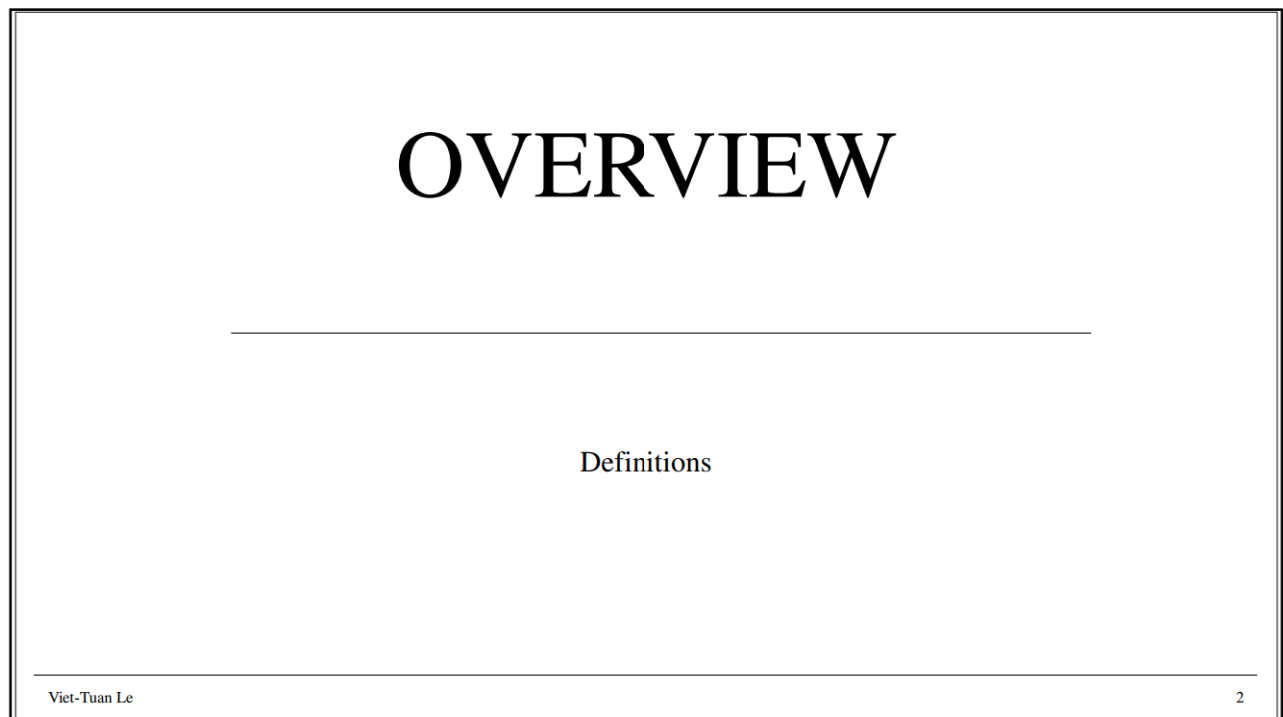




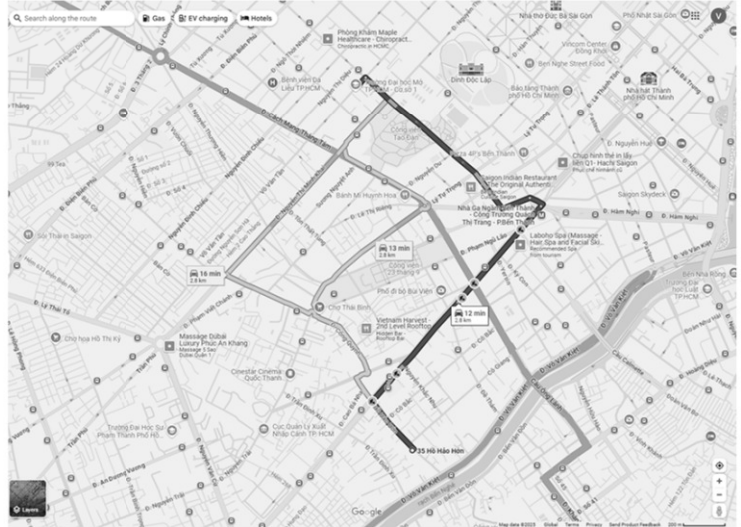
1



2

Application: route finding

- Objective: shortest?
fastest? most scenic?
- Actions: go straight, turn
left, turn right



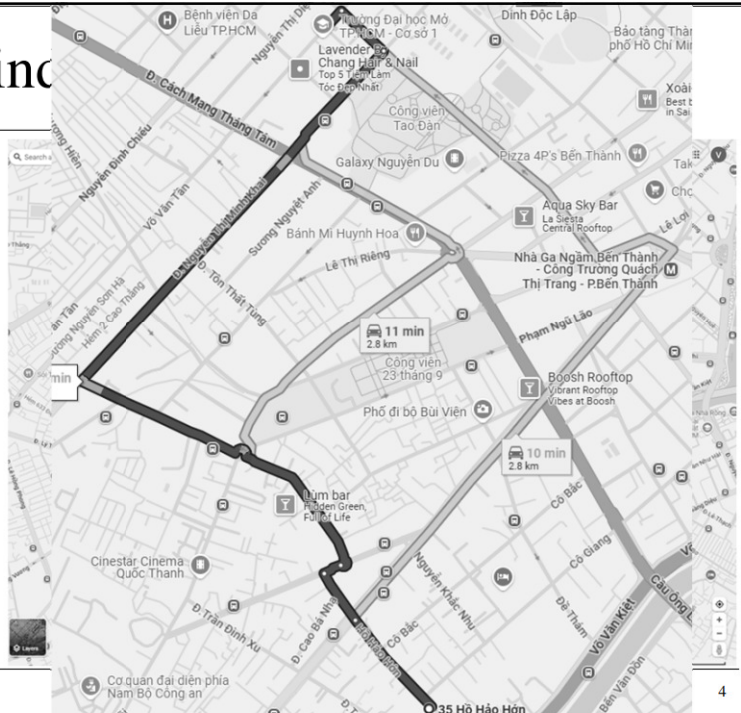
Viet-Tuan Le

3

3

Application: route find

- Objective: shortest?
fastest? most scenic?
- Actions: go straight, turn
left, turn right



Viet-Tuan Le

4

4

Application: robot motion planning

- Objective: fastest path
- Actions: acceleration and throttle



Viet-Tuan Le

5

Application: robot motion planning

- Objective: fastest? most energy efficient? safest? most expressive?
- Actions: translate and rotate joints



Viet-Tuan Le

6

6

Application: multi-robot systems

- Objective: fastest? most energy efficient?
- Actions: acceleration and steering of all robots

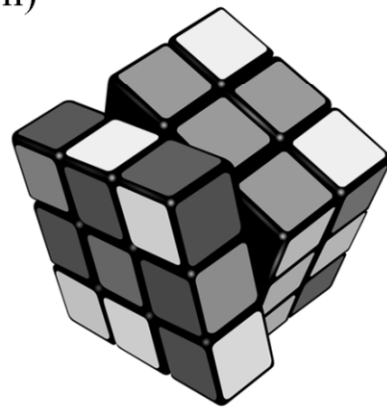
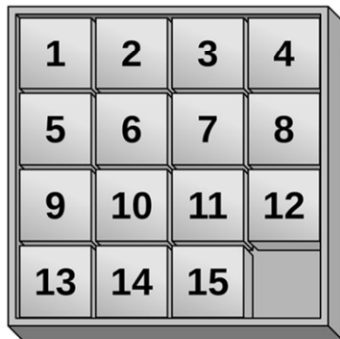


Viet-Tuan Le

7

Application: solving puzzles

- Objective: reach a certain configuration
- Actions: move pieces (e.g., Move12Down)



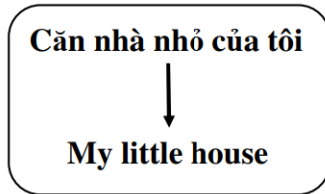
Viet-Tuan Le

8

8

Application: machine translation

- Objective: fluent English and preserves meaning
- Actions: append single words (e.g., the)



Beyond reflex

- Classifier (reflex-based models):

$$x \rightarrow \boxed{f} \rightarrow \text{single action } y \in \{-1, +1\}$$

- Search problem (state-based models):

$$x \rightarrow \boxed{f} \rightarrow \text{action sequence } (a_1, a_2, a_3, a_4, \dots)$$

Key: need to consider future consequences of an action!

MODELING

11

Transportation example



Example: transportation

Street with blocks numbered 1 to n .
Walking from s to $s + 1$ takes 1 minute.
Taking a magic tram from s to $2s$ takes 2 minutes.
How to travel from 1 to n in the least time?

[semi-live solution: TransportationProblem]

12

TREE SEARCH

Viet-Tuan Le

13

13

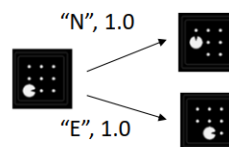
Search Problems

- A search problem consists of:

- A state space



- A successor function
(with actions, costs)



- A start state and a goal test
- A solution is a sequence of actions (a plan) which transforms the start state to a goal state

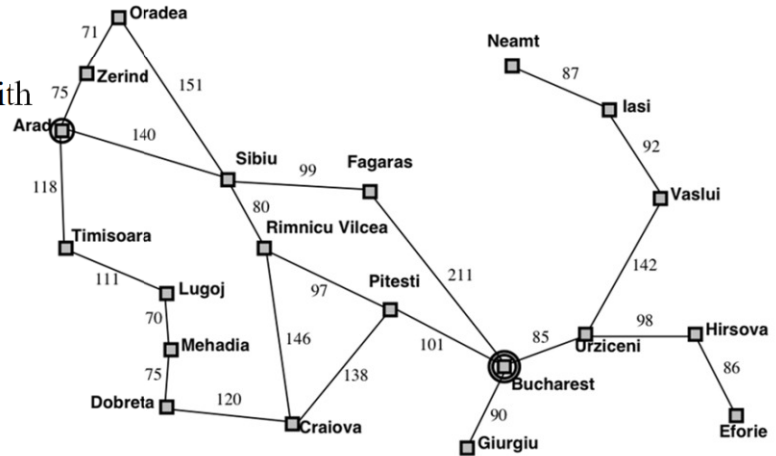
Viet-Tuan Le

14

14

Example: Traveling in Romania

- State space:
 - Cities
- Successor function:
 - Roads: Go to adjacent city with cost = distance
- Start state:
 - Arad
- Goal test:
 - Is state == Bucharest?
- Solution?



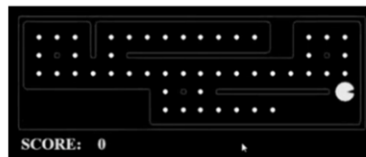
Viet-Tuan Le

15

15

What's in a State?

The world state includes every last detail of the environment



A search state keeps only the details needed for planning (abstraction)

- | | |
|---|--|
| <ul style="list-style-type: none"> • Problem: Pathing • States: • Actions: • Successor: • Goal test: | <ul style="list-style-type: none"> • Problem: Eat-All-Dots • States: • Actions: • Successor: • Goal test: |
|---|--|

Viet-Tuan Le

16

16

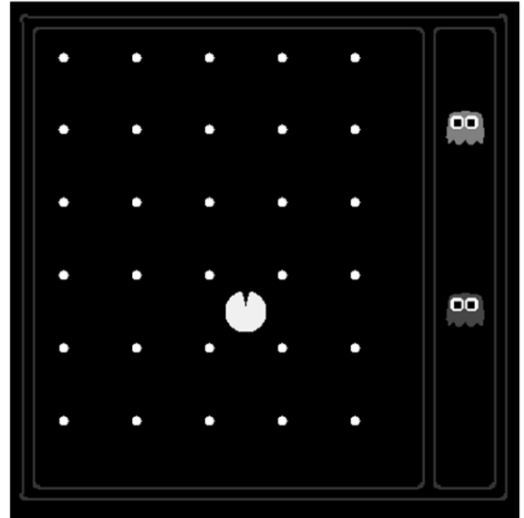
State Space Sizes?

- World state:

- Agent positions:
- Food count:
- Ghost positions:
- Agent facing:

- How many

- World states?
- States for pathing?
- States for eat-all-dots?

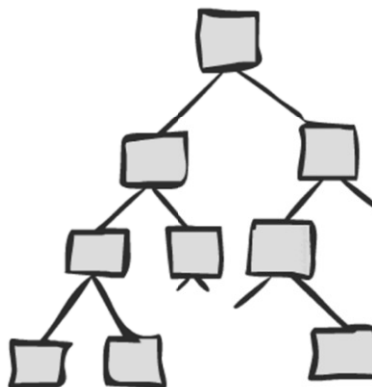


Viet-Tuan Le

17

17

State Space Graphs and Search Trees



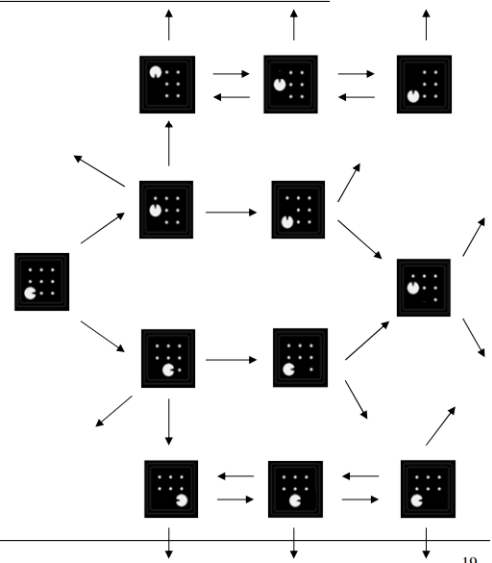
Viet-Tuan Le

18

18

State Space Graphs

- State space graph: A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



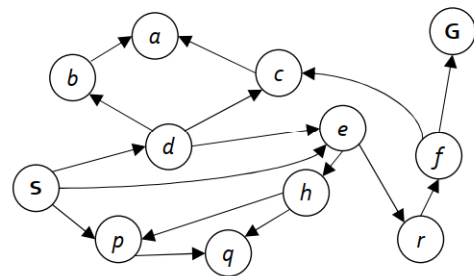
Viet-Tuan Le

19

19

State Space Graphs

- State space graph: A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



Tiny state space graph for a tiny search problem

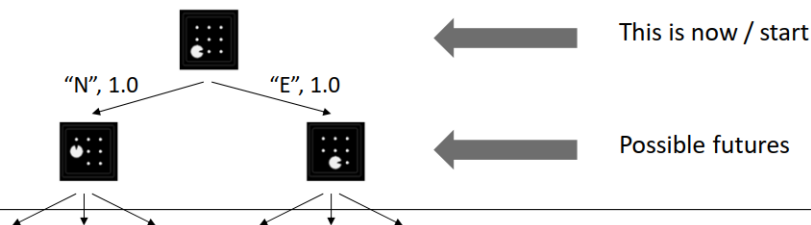
Viet-Tuan Le

20

20

Search Trees

- A search tree:
 - A “what if” tree of plans and their outcomes
 - The start state is the root node
 - Children correspond to successors
 - Nodes show states, but correspond to PLANS that achieve those states
 - For most problems, we can never actually build the whole tree

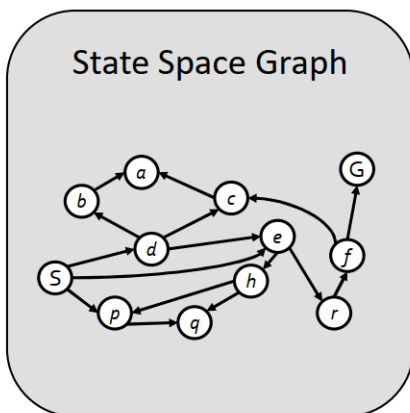


Viet-Tuan Le

21

21

State Space Graphs vs. Search Trees



Each NODE in in the search tree is an entire PATH in the state space graph.

We construct only what we need on demand

Search Tree

Viet-Tuan Le

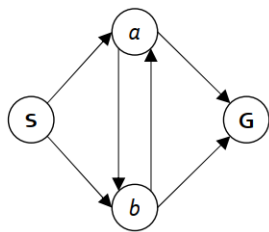
22

22

Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:

How big is its search tree (from S)?



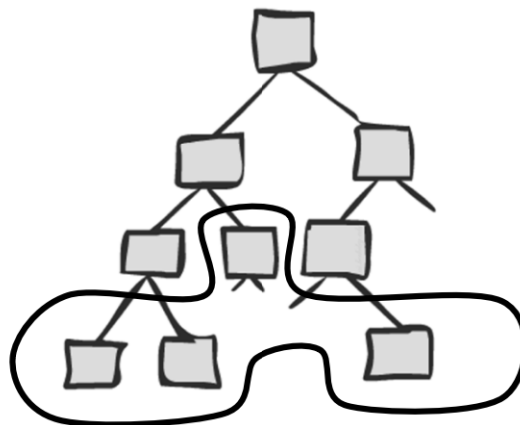
Important: Lots of repeated structure in the search tree!

Viet-Tuan Le

23

23

Tree Search

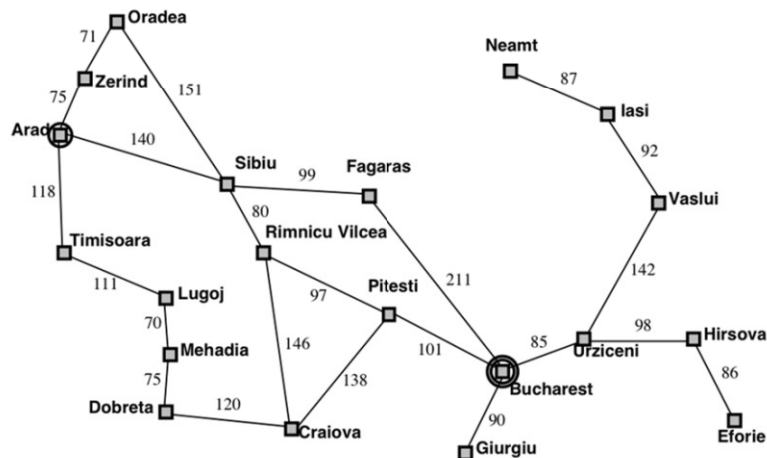


Viet-Tuan Le

24

24

Search Example: Romania



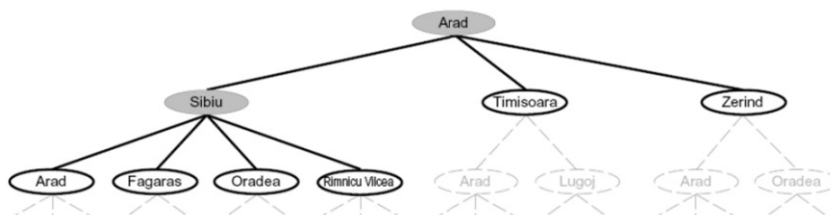
Viet-Tuan Le

25

25

Searching with a Search Tree

- Search:
 - Expand out potential plans (tree nodes)
 - Maintain a fringe of partial plans under consideration
 - Try to expand as few tree nodes as possible



Viet-Tuan Le

26

26

General Tree Search

- Important ideas:
 - Fringe
 - Expansion
 - Exploration strategy
- Main question: which fringe nodes to explore?

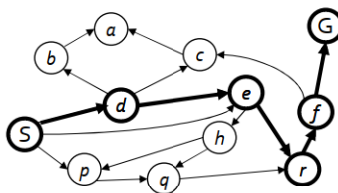
```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

Viet-Tuan Le

27

27

Example: Tree Search



Viet-Tuan Le

28

28

Depth-First Search



Viet-Tuan Le

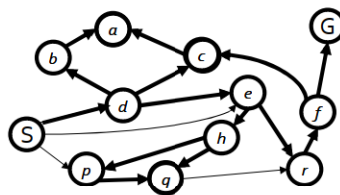
29

29

Depth-First Search

Strategy: expand a deepest node first

Implementation: Fringe is a LIFO stack

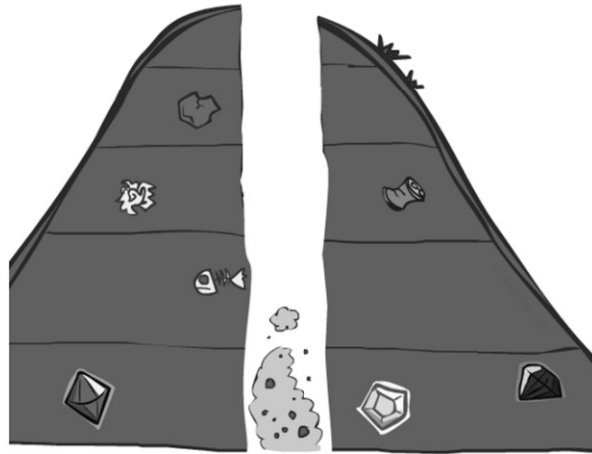


Viet-Tuan Le

30

30

Search Algorithm Properties



Viet-Tuan Le

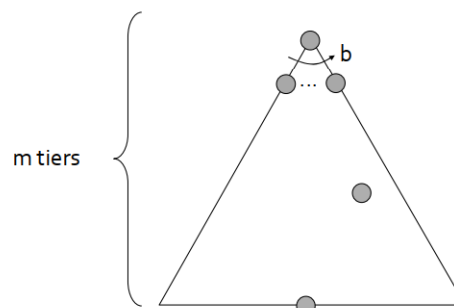
31

31

Search Algorithm Properties

- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?

- Cartoon of search tree:
 - b is the branching factor
 - m is the maximum depth
 - solutions at various depths



- Number of nodes in entire tree?

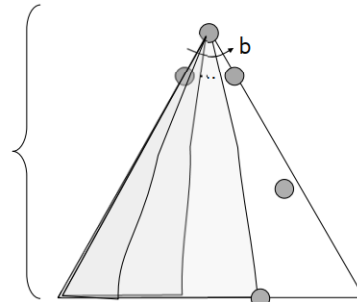
Viet-Tuan Le

32

32

Depth-First Search (DFS) Properties

- What nodes DFS expand?
 - Some left prefix of the tree.
 - Could process the whole tree!
 - If m is finite
- How much space does the fringe take?
 - Only has siblings on path to root, so
- Is it complete?
 - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
 - No, it finds the “leftmost” solution, regardless of depth or cost

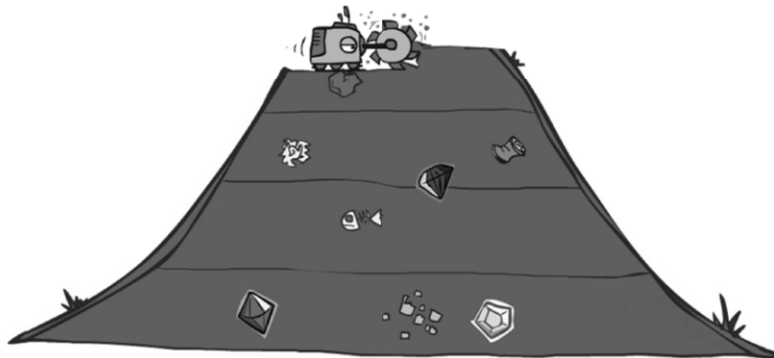


Viet-Tuan Le

33

33

Breadth-First Search



Viet-Tuan Le

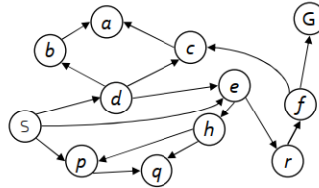
34

34

Breadth-First Search

Strategy: expand a shallowest node first

Implementation: Fringe is a FIFO queue



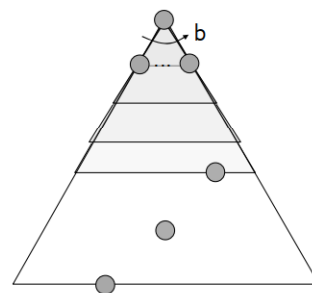
Viet-Tuan Le

35

35

Breadth-First Search (BFS) Properties

- What nodes does BFS expand?
 - Processes all nodes above shallowest solution
 - Let depth of shallowest solution be s
 - Search takes time s tiers
- How much space does the fringe take?
 - Has roughly the last tier
- Is it complete?
 - s must be finite if a solution exists, so yes!
- Is it optimal?
 - Only if costs are all 1 (more on costs later)



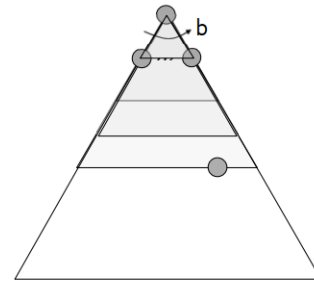
Viet-Tuan Le

36

36

Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
 - Run a DFS with depth limit 1. If no solution...
 - Run a DFS with depth limit 2. If no solution...
 - Run a DFS with depth limit 3.



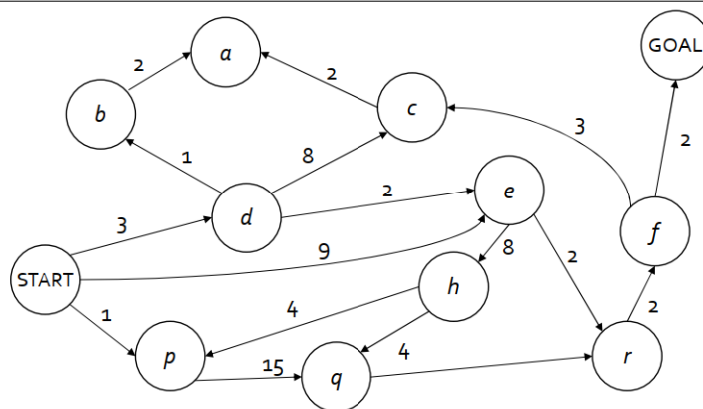
- Isn't that wastefully redundant?
 - Generally most work happens in the lowest level searched, so not so bad!

Viet-Tuan Le

37

37

Cost-Sensitive Search



BFS finds the shortest path in terms of number of actions.
It does not find the least-cost path. We will now cover a similar
algorithm which does find the least-cost path.

Viet-Tuan Le

38

38

Uniform Cost Search



Viet-Tuan Le

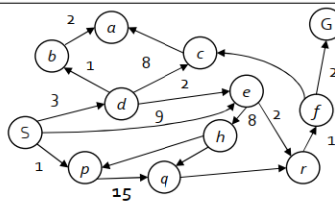
39

39

Uniform Cost Search

Strategy: expand a cheapest node first:

Fringe is a priority queue (priority: cumulative cost)



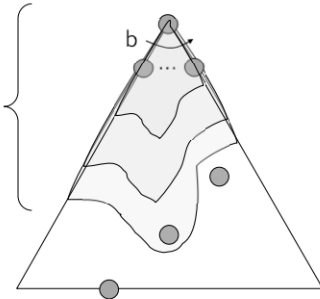
Viet-Tuan Le

40

40

Uniform Cost Search (UCS) Properties

- What nodes does UCS expand?
 - Processes all nodes with cost less than cheapest solution!
 - If that solution costs C^* and arcs cost at least ϵ , then the “effective depth” is roughly C^*/ϵ
 - Takes time $O(b^{C^*/\epsilon})$ (exponential in effective depth) C^*/ϵ “tiers”
- How much space does the fringe take?
 - Has roughly the last tier, so $O(b^{C^*/\epsilon})$
- Is it complete?
 - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
 - Yes! (Proof via A^*)



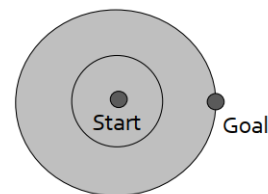
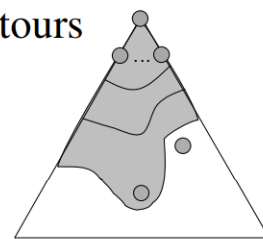
Viet-Tuan Le

41

41

Uniform Cost Issues

- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
- The bad:
 - Explores options in every “direction”
 - No information about goal location
- We’ll fix that soon!



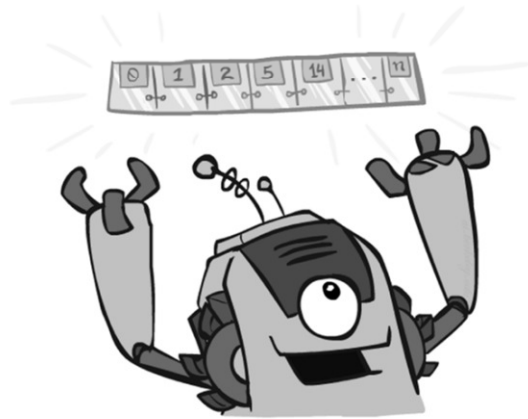
Viet-Tuan Le

42

42

The One Queue

- All these search algorithms are the same except for fringe strategies
 - DFS: Fringe is a Stack
 - BFS: Fringe is a Queue
 - UCS: Fringe is a PriorityQueue
 - Can even code one implementation that takes a variable queuing object



Q&A

Thank you!